

**Problem # 1:** Multiple choice question. [10 x 1pt]

- 1) Suppose the first 4 numbers in the Fibonacci sequence are given as 1,1,2,3. What is the eighth term of the Fibonacci sequence?
A. 13 B. 34 C. 8 D. 21
- 2) The worst-case time complexity of ----- is $O(n^2)$.
A. Merge sort B. Quick sort C. Binary search D. Sequential search
- 3) When does the worst case of Quicksort occur?
A. Array is already sorted in the same order
B. Array is already sorted in reverse order
C. All elements of the array are the same
D. All of the above
- 4) The solution of the recurrence in the table below is:

$F(n)=F(n-1)+3n, \quad T(1)=5$

- A. $3 * n + 2$ B. $n * \lg n$
C. $1.5 * n^2 + 1.5 * n + 2$ D. None of the above
- 5) Given the below sorted array:
 $arr[] = \{1, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59\}$
To determine if $x = 37$ exists in the above array using the binary search algorithm, the numbers compared in order with x are:
A. 19, 31, 47, 37 B. 23, 31, 43, 37
C. 23, 41, 31, 37 D. None of the above
- 6) Minimum number of moves required to solve a tower of Hanoi problem with n disks is -----:
A. 2^n B. n^2 C. $n^2 - 1$ D. $2^n - 1$
- 7) If the size of an array is 2048, then the number of comparisons done by binary search and sequential search is ----- respectively.
A. 12, 2048 B. 11, 2048 C. 2048, 12 D. 2048, 11

8) The big theta of

$$F(n) = 9F\left(\frac{n}{3}\right) + n$$

A. $n * \log n$

B. n^2

C. n

D. $\log n$

9) Which of the following is the correct order of growth (from the lowest to highest)?

A. $\Theta(\lg n), \Theta(n \lg n), \Theta(n^4), \Theta(n^n), \Theta(3^n), \Theta(n!)$

B. $\Theta(\lg n), \Theta(n \lg n), \Theta(n^4), \Theta(3^n), \Theta(n!), \Theta(n^n)$

C. $\Theta(\lg n), \Theta(n \lg n), \Theta(n^4), \Theta(n!), \Theta(3^n), \Theta(n^n)$

D. None of the above

10) In mergesort to merge two subarrays $U = [12, 18, 23, 29]$ and $V = [14, 17, 22, 31, 33]$ into one array S , how many times the basic operation (comparison) in merge algorithm that will be executed?

A. 5

B. 6

C. 7

D. None of the above

*** Write the correct answers of the problem # 1 in the table below:

<u>Q1</u>	<u>Q2</u>	<u>Q3</u>	<u>Q4</u>	<u>Q5</u>	<u>Q6</u>	<u>Q7</u>	<u>Q8</u>	<u>Q9</u>	<u>Q10</u>
D	B	D	C	C	D	A	B	B	C

Problem # 2: Answer the following questions for the given below recurrence equation

$$F(n) = 4F(n-1) - 3F(n-2)$$

1) What is the characteristic equation? [2pts]

Sol:

$$r^2 - 4 * r + 3 = 0$$

2) What are the roots of the characteristic equation? [2pts]

Sol:

$$(r-3) * (r-1) = 0$$

$$r1 = 3$$

$$r2 = 1$$

هذا السؤال يتكرر دائما بتسأل عن رقم بالسطر الاول بالجدول وانت بتعطيها الحواب من السطر الثاني

The [Fibonacci numbers](#) upto certain term can be represented as:

1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144.....

or

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144....

$$F[i]=F[i-1]+F[i-2]$$

$n =$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	...
$x_n =$	0	1	1	2	3	5	8	13	21	34	55	89	144	233	377	...

Establishing order of growth using the definition O-notation



Definition: $f(n)$ is in $O(g(n))$, denoted $f(n) \in O(g(n))$, if order of growth of $f(n) \leq$ order of growth of $g(n)$ (within constant multiple), i.e., there exist positive constant c and non-negative integer n_0 such that

$$f(n) \leq c g(n) \text{ for every } n \geq n_0$$

Ω -notation



- Formal definition
 - A function $t(n)$ is said to be in $\Omega(g(n))$, denoted $t(n) \in \Omega(g(n))$, if $t(n)$ is bounded below by some constant multiple of $g(n)$ for all large n , i.e., if there exist some positive constant c and some nonnegative integer n_0 such that
 $t(n) \geq cg(n)$ for all $n \geq n_0$

Ω -notation



- Formal definition
 - A function $t(n)$ is said to be in $\Omega(g(n))$, denoted $t(n) \in \Omega(g(n))$, if $t(n)$ is bounded below by some constant multiple of $g(n)$ for all large n , i.e., if there exist some positive constant c and some nonnegative integer n_0 such that
 $t(n) \geq cg(n)$ for all $n \geq n_0$

$$\Theta(\lg n) \quad \Theta(n) \quad \Theta(n \lg n) \quad \Theta(n^2) \quad \Theta(n^j) \quad \Theta(n^k) \quad \Theta(a^n) \quad \Theta(b^n) \quad \Theta(n!),$$

Solving Recurrences Using the Characteristic Equation



The Fibonacci numbers:

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

The Fibonacci recurrence:

$$F(n) = F(n-1) + F(n-2)$$

$$F(0) = 0$$

$$F(1) = 1$$

$$F(n) - F(n-1) - F(n-2) = 0$$

General 2nd order linear homogeneous recurrence with constant coefficients:

$$aX(n) + bX(n-1) + cX(n-2) = 0$$

Example : solve the recurrence



$$t(n)=3t(n-1)+4t(n-2) \quad t(0)=0 \quad t(1)=1$$

We solve the recurrence

$$\begin{aligned} t_n - 3t_{n-1} - 4t_{n-2} &= 0 & \text{for } n > 1 \\ t_0 &= 0 \\ t_1 &= 1 \end{aligned}$$

1. Obtain the characteristic equation:

$$\begin{aligned} t_n - 3t_{n-1} - 4t_{n-2} &= 0 \\ r^2 - 3r - 4 &= 0. \end{aligned}$$

2. Solve the characteristic equation:

$$r^2 - 3r - 4 = (r - 4)(r + 1) = 0.$$

The roots are $r = 4$ and $r = -1$.

3. Apply [Theorem B.1](#) to get the general solution to the recurrence:

$$t_n = c_1 4^n + c_2 (-1)^n.$$

Decrease-by-a-constant-factor recurrences – The Master Theorem



$$T(n) = aT(n/b) + f(n), \quad \text{where } f(n) \in \Theta(n^k), \quad k \geq 0$$

1. $a < b^k$ $T(n) \in \Theta(n^k)$
2. $a = b^k$ $T(n) \in \Theta(n^k \log n)$
3. $a > b^k$ $T(n) \in \Theta(n^{\log_b a})$

Examples:

- $T(n) = T(n/2) + 1$ $\Theta(\log n)$
- $T(n) = 2T(n/2) + n$ $\Theta(n \log n)$
- $T(n) = 3T(n/2) + n$ $\Theta(n^{\log_2 3})$
- $T(n) = T(n/2) + n$ $\Theta(n)$

Summary :



1) linear homogeneous recurrence

⌘ 2nd order

$$aX(n) + bX(n-1) + cX(n-2) = 0$$

Solving Recurrences Using the Characteristic Equation

Example:

$$F(n) = F(n-1) + F(n-2)$$

$$F(0) = 0$$

$$F(1) = 1 \quad \text{--->}$$

$$F(n) - F(n-1) - F(n-2) = 0$$



2) linear nonhomogeneous recurrence

⌘ Decrease-by-one Recurrences

$$T(n) = T(n-1) + f(n) \quad f(n) \neq 0$$

a) $T(n) = T(n-1) + c \quad T(1) = d \quad \rightarrow \quad f(n)=c$

b) **Solution :**

$$T(n) = (n-1)c + d \quad \text{linear}$$

b) $T(n) = T(n-1) + c n \quad T(1) = d \quad \rightarrow \quad f(n)=cn$

Solution:

$$T(n) = [n(n+1)/2 - 1] c + d \quad \text{quadratic}$$

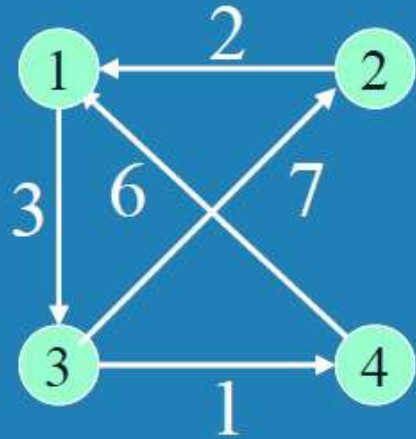
The binomial coefficient



▣ Definition

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \quad \text{for } 0 \leq k \leq n$$

Floyd's Algorithm (example)



$$D^{(0)} =$$

0	∞	3	∞
2	0	∞	∞
∞	7	0	1
6	∞	∞	0

$$D^{(1)} =$$

0	∞	3	∞
2	0	5	∞
∞	7	0	1
6	∞	9	0

$$D^{(2)} =$$

0	∞	3	∞
2	0	5	∞
9	7	0	1
6	∞	9	0

$$D^{(3)} =$$

0	10	3	4
2	0	5	6
9	7	0	1
6	16	9	0

$$D^{(4)} =$$

0	10	3	4
2	0	5	6
7	7	0	1
6	16	9	0

$$D^{(k)}[i,j] = \min \{D^{(k-1)}[i,j], D^{(k-1)}[i,k] + D^{(k-1)}[k,j]\}$$

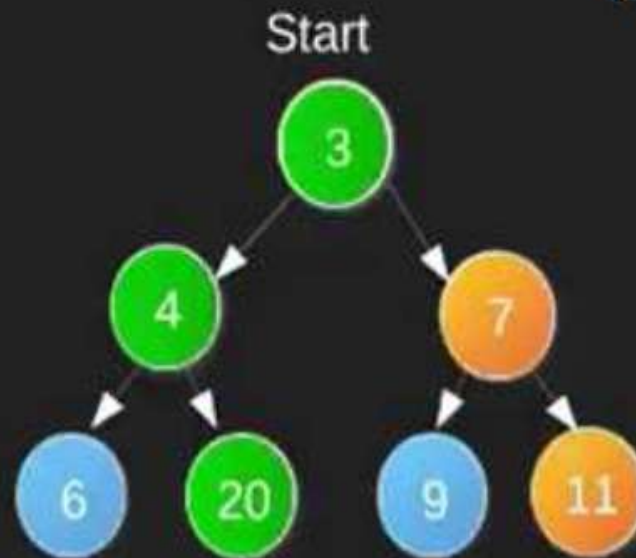
Basic components in Greedy approach

- ⑦ A ***selection procedure*** chooses the next item to add to the set. The selection is performed according to a greedy criterion that satisfies some locally optimal consideration at the time.
- ⑦ A ***feasibility check*** determines if the new set is feasible by checking whether it is possible to complete this set in such a way as to give a solution to the instance.
- ⑦ A ***solution check*** determines whether the new set constitutes a solution to the instance.

Greedy (Often) does not work

Problem : Find the branch that give largest sum

Greedy Answer: $3+7+11=21$



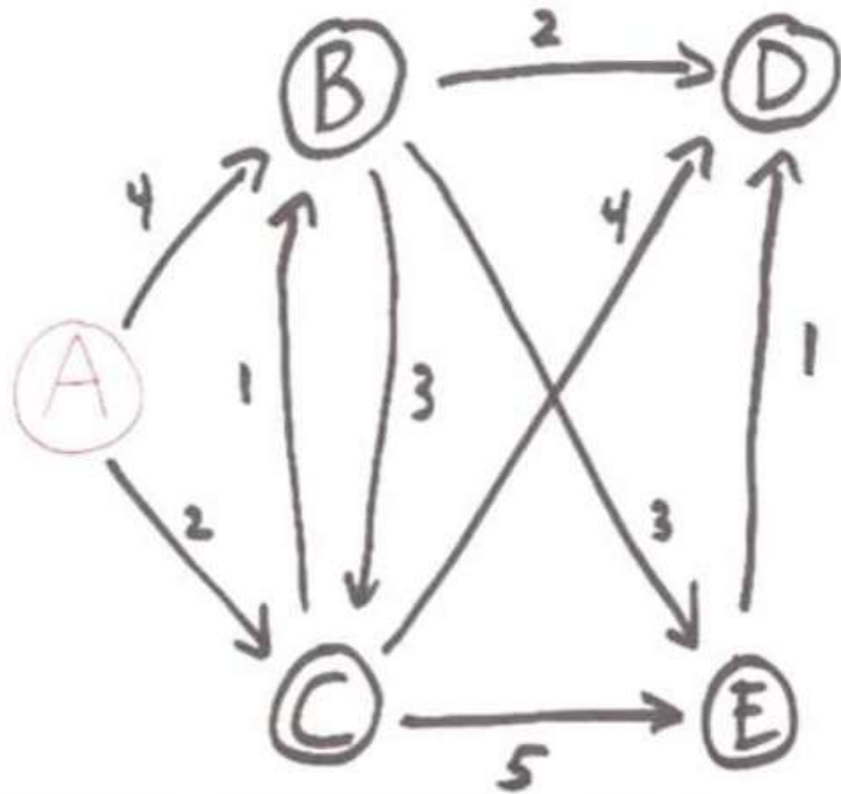
Actual Answer :
 $\Rightarrow 3+4+20 = \underline{27}$

● : Actual Choice

● : Greedy Choice



Dijkstra's algorithm example



	A	B	C	D	E
A	0	∞	∞	∞	∞
C	0	4	2	∞	∞
B	0	3	2	6	7
D	0	5	2	5	6
E	0	3	2	5	6

A C B D
→

A: 0
B: 3
C: 2
D: 5
E: 6

An example

Character	Frequency	C1(Fixed-Length)	C2	C3 (Huffman)
<i>a</i>	16	000	10	00
<i>b</i>	5	001	11110	1110
<i>c</i>	12	010	1110	110
<i>d</i>	17	011	110	01
<i>e</i>	10	100	11111	1111
<i>f</i>	25	101	0	10

Table 4.1 • Three codes for the same file. C3 is optimal.

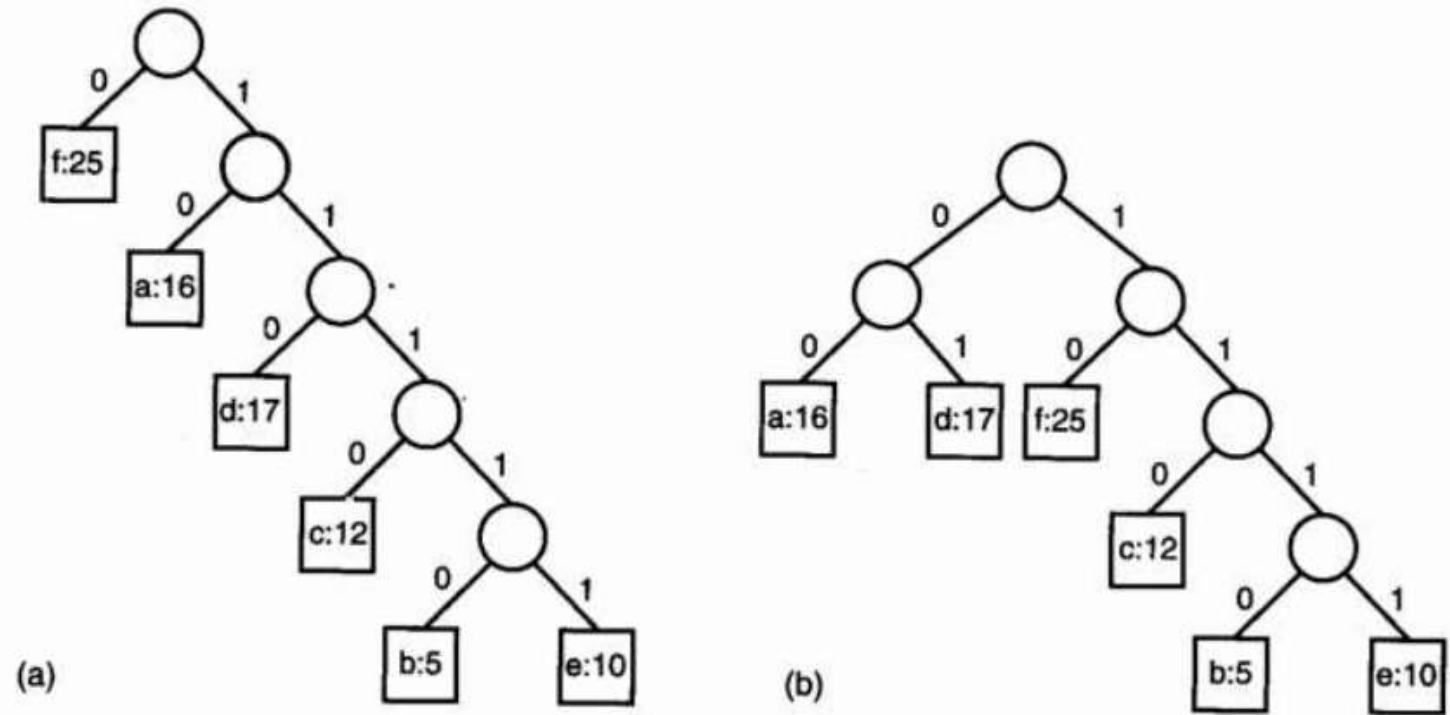


Figure 4.10 • The binary character code for Code C2 in Example 4.7 appears in (a), while the one for Code C3 (Huffman) appears in (b).

Suppose there are three jobs and the service times for these jobs are

$$t_1 = 5, \quad t_2 = 10, \quad \text{and} \quad t_3 = 4.$$

The actual time units are not relevant to the problem. If we schedule them in the order 1, 2, 3, the times spent in the system for the three jobs are as follows:

Job	Time in the System
1	5 (service time)
2	5 (wait for job 1) + 10 (service time)
3	5 (wait for job 1) + 10 (wait for job 2) + 4 (service time)

The total time in the system for this schedule is

$$\underbrace{5}_{\text{Time for job 1}} + \underbrace{(5 + 10)}_{\text{Time for job 2}} + \underbrace{(5 + 10 + 4)}_{\text{Time for job 3}} = 39.$$

This same method of computation yields the following list of all possible schedules and total times in the system:

Schedule	Total Time in the System
[1, 2, 3]	$5 + (5 + 10) + (5 + 10 + 4) = 39$
[1, 3, 2]	$5 + (5 + 4) + (5 + 4 + 10) = 33$
[2, 1, 3]	$10 + (10 + 5) + (10 + 5 + 4) = 44$
[2, 3, 1]	$10 + (10 + 4) + (10 + 4 + 5) = 43$
[3, 1, 2]	$4 + (4 + 5) + (4 + 5 + 10) = 32$
[3, 2, 1]	$4 + (4 + 10) + (4 + 10 + 5) = 37$

Schedule [3, 1, 2] is optimal with a total time of 32.

بيجي سؤال backtracking على

هذا القانون تطبق مباشر

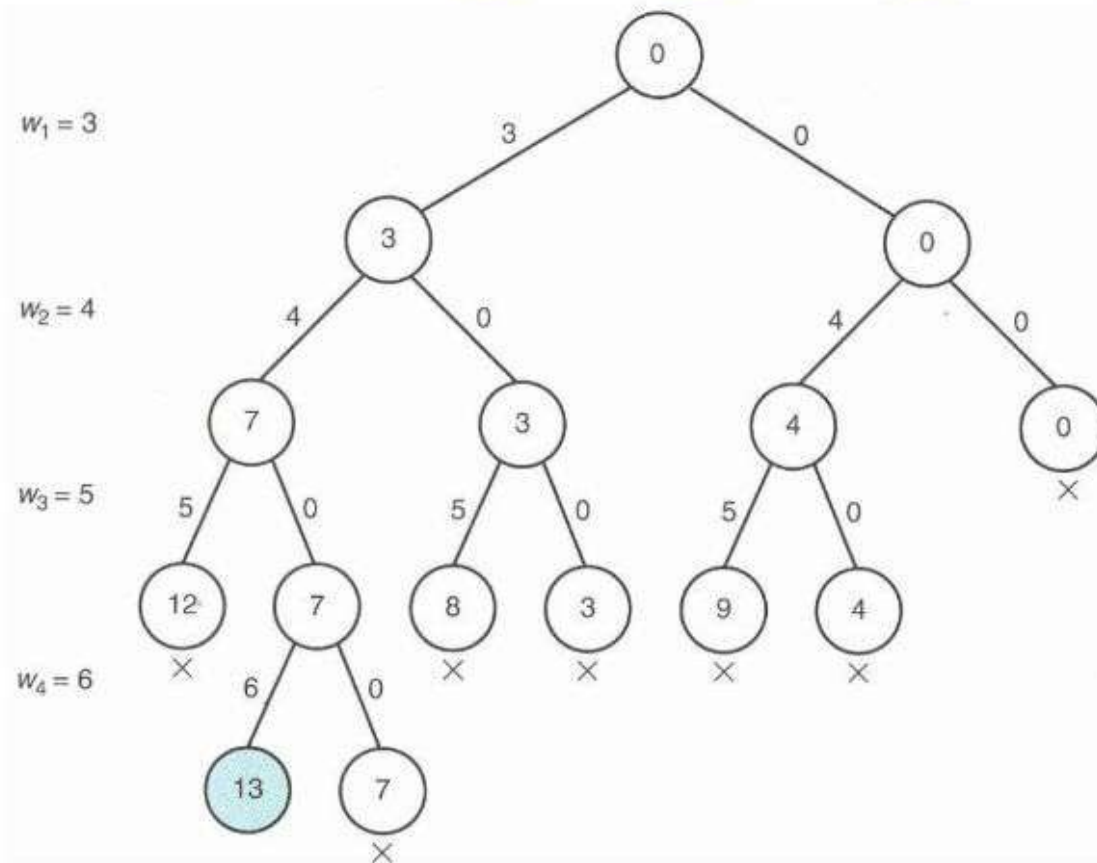
Efficiency

- ⑦ Checking the entire state space tree (number of nodes checked)

$$1 + n + n^2 + n^3 + \dots + n^n = \frac{n^{n+1} - 1}{n - 1}.$$

بيجي سؤال رسمة زي هيك وبسأل
 شو هم ال promising node
 وشو هم ال non promising

When $W = 13$ and $w_1 = 3, w_2 = 4, w_3 = 5, w_4 = 6$



To check whether a node is promising

- ⑦ Sort the weights in nondecreasing order
- ⑦ To check the node at level i
 - ④ $weight + w_{i+1} > W$
 - ④ $weight + total < W$